R/C Routines for Symmetric Matrices in Lower Triangular Column-Major Storage

Jan de Leeuw

Version June 27, 2018

Abstract

dodo

Contents

1 Introduction **1**

2 Code Principles **2**

3 Code **2**

3.1 R code 2

3.2 C Code 4

4 References **17**

Note: This is a working paper which will be expanded/updated frequently. All suggestions for improvement are welcome. The directory gifi.stat.ucla.edu/smatrix has a pdf version, the bib file, the R and C code, and the complete Rmd file.

1 Introduction

Suppose A is a symmetric matrix, say

```
##     2     3     4     5     6
##     3     4     5     6     7
##     4     5     6     7     8
##     5     6     7     8     9
##     6     7     8     9    10
```

We store the elements on the matrix as a vector, using the lower triangle (including the diagonal) with the first column first, then the second column, and so on. In R this is simply

```
a[outer(1:5,1:5,">=")]
```

```
##  [1]  2  3  4  5  6  4  5  6  7  6  7  8  8  9 10
```

This is the symmetric matrix A in LTCM (lower triangle column-major) storage mode. Our job in this paper is to rewrite some of the basic linear algebra algorithms for symmetric matrices so that they apply to LTCM matrices. Clearly this implies we save memory, but since acsessing elements will be more complicated we sacrifice some speed.

2 Code Principles

Our R code are wrappers for C routines. Since we want to be able to use our C routines outisde the R environment as well, they should use a few of the R resources as possible. Thus we use the .C() interface, and we try to avoid memory allocation and I/O as much as possible. All C routines return void, and pass by reference (take pointers as arguments).

To find element (i, j) in a matrix we use C routines `MINDEX, TINDEX, SINDEX` that compute the location of the element in the linear storage. `MINDEX` is for full matrices, `TINDEX` for LTCM matries, and `SINDEX` for strictly lower triangular matrices. In our routines roiw and column indices start at one, not at zero. Thus we also use `VINDEX` to access elements of a vector, where in C `a[VINDEX(i)] = a[i-1]`. To complete the set we also have `UINDEX`, which is used for a sequence of LTCM matrices in linear storage. Because these routines are used heavily, we define them to be static inline in the header file `smatrix.h`. as few R

3 Code

3.1 R code

3.1.1 smatrix.R

```
dyn.load("smatrix.so")

scholesky <- function (a, pivot = FALSE, eps = 1e-10) {
  m <- length (a)
  n <- (sqrt (1 + 8 * m) - 1) / 2
  h <- .C(
    "pcholesky",
    n = as.integer(n),
    a = as.double (a),
    rank = as.integer(0),
    order = as.integer(1:n),
    pivot = as.integer(pivot),
    eps = as.double(eps)
  )
  return (h)
```

```r
}

triangle2matrix <- function (x) {
  m <- length (x)
  n <- round ((sqrt (1 + 8 * m) - 1) / 2)
  h <-
    .C("trimat", as.integer (n), as.double (x), as.double (rep (0, n * n)))
  return (matrix(h[[3]], n, n))
}

matrix2triangle <- function (x) {
  n <- dim(x)[1]
  m <- n * (n + 1) / 2
  h <-
    .C("mattri", as.integer (n), as.double (x), as.double (rep (0, m)))
  return (h[[3]])
}

triangle2triangle <- function (x) {
  m <- length (x)
  n <- round ((sqrt (1 + 8 * m) - 1) / 2)
  h <-
    .C("tritri", as.integer (n), as.double (x), as.double (rep (0, n * n)))
  return (matrix(h[[3]], n, n))
}

matrix2print <- function (x, w = 15, p = 10) {
  n <- nrow (x)
  m <- ncol (x)
  h <-
    .C(
      "primat",
      as.integer(n),
      as.integer(m),
      as.integer(w),
      as.integer(p),
      as.double (x)
    )
}
```

3.2 C Code

3.2.1 smatrix.h

```c
#ifndef SMATRIX_H
#define SMATRIX_H

#include <math.h>
#include <stdbool.h>
#include <stdio.h>
#include <stdlib.h>

#ifdef USING_R
#include <R.h>
#endif

static inline int VINDEX(const int);
static inline int MINDEX(const int, const int, const int);
static inline int SINDEX(const int, const int, const int);
static inline int TINDEX(const int, const int, const int);
static inline int AINDEX(const int, const int, const int, const int, const int);
static inline int UINDEX(const int, const int, const int, const int, const int);

static inline double SQUARE(const double);
static inline double THIRD(const double);
static inline double FOURTH(const double);
static inline double FIFTH(const double);

static inline double MAX(const double, const double);
static inline double MIN(const double, const double);
static inline int IMIN(const int, const int);
static inline int IMAX(const int, const int);

// index function for vector

static inline int VINDEX(const int i) { return i - 1; }

// index function for rectangular matrix

static inline int MINDEX(const int i, const int j, const int n) {
  return (i - 1) + (j - 1) * n;
}

// index function for symmetric matrix
```

```c
static inline int AINDEX(const int i, const int j, const int k, const int n,
                         const int m) {
  return (i - 1) + (j - 1) * n + (k - 1) * n * m;
}

// index function for lower triangle without diagonal

static inline int SINDEX(const int i, const int j, const int n) {
  return ((j - 1) * n) - (j * (j - 1) / 2) + (i - j) - 1;
}

// index function for lower triangle with diagonal

static inline int TINDEX(const int i, const int j, const int n) {
  return ((j - 1) * n) - ((j - 1) * (j - 2) / 2) + (i - (j - 1)) - 1;
}

// index function for multiple lower triangle with diagonal

static inline int UINDEX(const int i, const int j, const int k, const int n,
                         const int m) {
  return ((k - 1) * n * (n + 1) / 2) + ((j - 1) * n) - ((j - 1) * (j - 2) / 2) +
         (i - (j - 1)) - 1;
}

static inline double SQUARE(const double x) { return x * x; }
static inline double THIRD(const double x) { return x * x * x; }
static inline double FOURTH(const double x) { return x * x * x * x; }
static inline double FIFTH(const double x) { return x * x * x * x * x; }

static inline double MAX(const double x, const double y) {
  return (x > y) ? x : y;
}

static inline double MIN(const double x, const double y) {
  return (x < y) ? x : y;
}

static inline int IMAX(const int x, const int y) { return (x > y) ? x : y; }

static inline int IMIN(const int x, const int y) { return (x < y) ? x : y; }


#endif /* SMATRIX_H */
```

3.2.2 smatrix.c

```c
#define USING_R

#include "smatrix.h"
#include "sroutines.h"

/*
 * The routine scholesky() replaces a positive definite matrix A of order n
 * in lower-triangular column-major storage by its Cholesky factor L. Thus
 * A = LL'. The determinent of A is returned as well.
 *
 * If a pivot is zero the routine jumps over the pivot and continues,
 * and sets the singularity warning. Zero of a pivot means less than a small
 * positive eps.
 *
 * If a pivot is negative the routine stops, and sets the indefinite warning.
 * Negativity of a pivot means less than -eps.
 */

void scholesky(const int *nn, double *a, bool *singular, bool *indefinite,
               double *det, const double *eps) {
  int n = *nn;
  double t = 0.0, s = 0.0, dt = 1.0;
  for (int k = 1; k <= n; k++) {
    t = a[TINDEX(k, k, n)];
    if (t < -*eps) {
      *indefinite = true;
      return;
    }
    if (t < *eps) {
      *singular = true;
      continue;
    }
    dt *= t;
    s = sqrt(t);
    for (int i = k; i <= n; i++) {
      a[TINDEX(i, k, n)] = a[TINDEX(i, k, n)] / s;
    }
    for (int i = k + 1; i <= n; i++) {
      for (int j = k + 1; j <= i; j++) {
```

```
        a[TINDEX(i, j, n)] -= a[TINDEX(i, k, n)] * a[TINDEX(j, k, n)];
      }
    }
  }
  *det = dt;
  return;
}

/*
 * The routine pcholesky() replaces a pivoted version of a positive
 * semi-definite matrix A of order n in lower-triangular column-major storage by
 * its Cholesky factor L. Thus we find a permutation P of rows and columns of A
 * such that P'AP=LL'.
 *
 * Pivoting is on the maximum diagonal element. If this is negative the matrix
 * was not positive semidefinite and we return the indefinite flag.  Negatie for
 * a pivot means less than minus a small positive eps. The approximate rank is
 * returned as the number of nonzero pivots, and the dterminant is returned as
 * the product of pivots. On exit the vector order has the pivot permutation.
 */

void pcholesky(const int *nn, double *a, double *det, int *rank, int *order,
               bool *indefinite, const double *eps) {
  int n = *nn, q = 0, itp = 0;
  double t = 0.0, at = 0.0, tmp = 0.0, tp1 = 0.0, tp2 = 0.0, tp3 = 0.0, s = 0.0,
         dt = 1.0;
  *rank = 0;
  for (int i = 1; i <= n; i++) {
    order[VINDEX(i)] = i;
  }
  for (int k = 1; k <= n; k++) {
    t = 0.0;
    q = 0;
    for (int i = k; i <= n; i++) {
      at = a[TINDEX(i, i, n)];
      if (at >= t) {
        t = at;
        q = i;
      }
    }
    if (t < -*eps) {
      *indefinite = true;
      return;
    }
```

```
    (*rank)++;
    dt /= t;
    itp = order[VINDEX(k)];
    order[VINDEX(k)] = order[VINDEX(q)];
    order[VINDEX(q)] = itp;
    tp1 = a[TINDEX(k, k, n)];
    tp2 = a[TINDEX(q, q, n)];
    tp3 = a[TINDEX(q, k, n)];
    for (int i = 1; i <= k - 1; i++) {
      tmp = a[TINDEX(k, i, n)];
      a[TINDEX(k, i, n)] = a[TINDEX(q, i, n)];
      a[TINDEX(q, i, n)] = tmp;
    }
    for (int i = k + 1; i <= q - 1; i++) {
      tmp = a[TINDEX(i, k, n)];
      a[TINDEX(i, k, n)] = a[TINDEX(q, i, n)];
      a[TINDEX(q, i, n)] = tmp;
    }
    for (int i = q + 1; i <= n; i++) {
      tmp = a[TINDEX(i, q, n)];
      a[TINDEX(i, q, n)] = a[TINDEX(i, k, n)];
      a[TINDEX(i, k, n)] = tmp;
    }
    a[TINDEX(q, q, n)] = tp1;
    a[TINDEX(k, k, n)] = tp2;
    a[TINDEX(q, k, n)] = tp3;
    s = sqrt(t);
    for (int i = k; i <= n; i++) {
      a[TINDEX(i, k, n)] /= s;
    }
    for (int i = k + 1; i <= n; i++) {
      for (int j = k + 1; j <= i; j++) {
        a[TINDEX(i, j, n)] -= a[TINDEX(i, k, n)] * a[TINDEX(j, k, n)];
      }
    }
  }
  *det = dt;
  return;
}

void primat(const int *n, const int *m, const int *w, const int *p,
            const double *x) {
  for (int i = 1; i <= *n; i++) {
    for (int j = 1; j <= *m; j++) {
```

```c
#ifdef USING_R
      Rprintf(" %*.*f ", *w, *p, x[MINDEX(i, j, *n)]);
#else
      printf(" %*.*f ", *w, *p, x[MINDEX(i, j, *n)]);
#endif
    }
#ifdef USING_R
    Rprintf("\n");
#else
    printf("\n");
#endif
  }
#ifdef USING_R
  Rprintf("\n\n");
#else
  printf("\n\n");
#endif
  return;
}

void pritru(const int *n, const int *w, const int *p, const double *x) {
  for (int i = 1; i <= *n; i++) {
    for (int j = 1; j <= i; j++) {
#ifdef USING_R
      Rprintf(" %*.*f ", *w, *p, x[TINDEX(i, j, *n)]);
#else
      printf(" %*.*f ", *w, *p, x[TINDEX(i, j, *n)]);
#endif
    }
#ifdef USING_R
    Rprintf("\n");
#else
    printf("\n");
#endif
  }
#ifdef USING_R
  Rprintf("\n\n");
#else
  printf("\n\n");
#endif
  return;
}

void prisru(const int *n, const int *m, const int *w, const int *p,
```

```
            const double *x) {
  for (int k = 1; k <= *m; k++) {
    for (int i = 1; i <= *n; i++) {
      for (int j = 1; j <= i; j++) {
#ifdef USING_R
        Rprintf(" %*.*f ", *w, *p, x[UINDEX(i, j, k, *n, *m)]);
#else
        printf(" %*.*f ", *w, *p, x[UINDEX(i, j, k, *n, *m)]);
#endif
      }
#ifdef USING_R
      Rprintf("\n");
#else
      printf("\n");
#endif
    }
#ifdef USING_R
    Rprintf("\n\n");
#else
    printf("\n\n");
#endif
  }
  return;
}

void trimat(const int *n, const double *x, double *y) {
  int nn = *n;
  for (int i = 1; i <= nn; i++) {
    for (int j = 1; j <= nn; j++) {
      y[MINDEX(i, j, nn)] =
          (i >= j) ? x[TINDEX(i, j, nn)] : x[TINDEX(j, i, nn)];
    }
  }
  return;
}

void tritri(const int *n, const double *x, double *y) {
  int nn = *n;
  for (int i = 1; i <= nn; i++) {
    for (int j = 1; j <= nn; j++) {
      y[MINDEX(i, j, nn)] = (i >= j) ? x[TINDEX(i, j, nn)] : 0.0;
    }
  }
  return;
```

```
}

void mattri(const int *n, const double *x, double *y) {
  int nn = *n;
  for (int j = 1; j <= nn; j++) {
    for (int i = j; i <= nn; i++) {
      y[TINDEX(i, j, nn)] = x[MINDEX(i, j, nn)];
    }
  }
  return;
}

void invtri(const int *nn, double *x, const double *eps, bool *ierr) {
  int n = *nn;
  double s = 0.0, t = 0.0;
  for (int k = 1; k <= n; k++) {
    t = x[TINDEX(k, k, n)];
    if (fabs(t) < *eps) {
      *ierr = true;
      return;
    }
    x[TINDEX(k, k, n)] = 1.0 / t;
    for (int i = k + 1; i <= n; i++) {
      s = 0.0;
      for (int j = k; j < i; j++) {
        s += x[TINDEX(i, j, n)] * x[TINDEX(j, k, n)];
      }
      x[TINDEX(i, k, n)] = -s / x[TINDEX(i, i, n)];
    }
  }
  return;
}

void ssweep(const int *nn, double *x, double *det, bool *singular,
            const double *eps) {
  int n = *nn;
  double dg = 0.0, de = 1.0;
  *singular = false;
  *det = 1.0;
  for (int k = 1; k <= n; k++) {
    dg = x[TINDEX(k, k, n)];
    if (dg < *eps) {
      *singular = true;
      continue;
```

```
    }
    de *= dg;
    x[TINDEX(k, k, n)] = -1 / dg;
    for (int j = 1; j <= k - 1; j++) {
      for (int i = j; i <= k - 1; i++) {
        x[TINDEX(i, j, n)] -= x[TINDEX(k, i, n)] * x[TINDEX(k, j, n)] / dg;
      }
    }
    for (int j = k + 1; j <= n; j++) {
      for (int i = j; i <= n; i++) {
        x[TINDEX(i, j, n)] -= x[TINDEX(i, k, n)] * x[TINDEX(j, k, n)] / dg;
      }
    }
    for (int j = 1; j <= k - 1; j++) {
      for (int i = k + 1; i <= n; i++) {
        x[TINDEX(i, j, n)] -= x[TINDEX(i, k, n)] * x[TINDEX(k, j, n)] / dg;
      }
    }
    for (int j = 1; j <= k - 1; j++) {
      x[TINDEX(k, j, n)] /= dg;
    }
    for (int j = k + 1; j <= n; j++) {
      x[TINDEX(j, k, n)] /= dg;
    }
  }
  *det = de;
  return;
}

void smjacobi(const int *nn, const int *mm, double *a, double *evec,
              double *fini, double *f, int *itel, const int *itmax,
              const double *eps, const bool *verbose, const bool *vectors) {
  int n = *nn, m = *mm;
  double d = 0.0, cost = 0.0, sint = 0.0, u = 0.0, p = 0.0, q = 0.0, r = 0.0,
         piil = 0.0, pijl = 0.0, lbd = 0.0, dd = 0.0, pp = 0.0, dp = 0.0;
  double fold = 0.0, fnew = 0.0, oldi = 0.0, oldj = 0.0;
  *itel = 1;
  if (*vectors) {
    for (int i = 1; i <= n; i++) {
      for (int j = 1; j <= n; j++) {
        evec[MINDEX(i, j, n)] = (i == j) ? 1.0 : 0.0;
      }
    }
  }
```

```
for (int k = 1; k <= m; k++) {
  for (int i = 1; i <= n; i++) {
    fold += SQUARE(a[UINDEX(i, i, k, n, m)]);
  }
}
*fini = fold;
while (true) {
  for (int j = 1; j <= n - 1; j++) {
    for (int i = j + 1; i <= n; i++) {
      dd = 0.0, pp = 0.0, dp = 0.0;
      for (int k = 1; k <= m; k++) {
        p = a[UINDEX(i, j, k, n, m)];
        q = a[UINDEX(i, i, k, n, m)];
        r = a[UINDEX(j, j, k, n, m)];
        d = (q - r) / 2.0;
        dd += SQUARE(d);
        pp += SQUARE(p);
        dp += p * d;
      }
      lbd = ((dd + pp) - sqrt(SQUARE(dd - pp) + 4.0 * SQUARE(dp))) / 2.0;
      u = dp / sqrt(SQUARE(dp) + SQUARE(lbd - pp));
      if ((fabs(dp) < 1e-15) && (pp <= dd)) {
        continue;
      }
      cost = sqrt((1 + u) / 2);
      sint = -sqrt((1 - u) / 2);
      if (*vectors) {
        for (int l = 1; l <= n; l++) {
          piil = evec[MINDEX(l, i, n)];
          pijl = evec[MINDEX(l, j, n)];
          evec[MINDEX(l, i, n)] = cost * piil - sint * pijl;
          evec[MINDEX(l, j, n)] = sint * piil + cost * pijl;
        }
      }
      for (int k = 1; k <= m; k++) {
        p = a[UINDEX(i, j, k, n, m)];
        q = a[UINDEX(i, i, k, n, m)];
        r = a[UINDEX(j, j, k, n, m)];
        for (int l = 1; l <= n; l++) {
          if ((l == i) || (l == j))
            continue;
          int il = IMIN(i, l);
          int li = IMAX(i, l);
          int jl = IMIN(j, l);
```

```
            int lj = IMAX(j, l);
            oldi = a[UINDEX(li, il, k, n, m)];
            oldj = a[UINDEX(lj, jl, k, n, m)];
            a[UINDEX(li, il, k, n, m)] = cost * oldi - sint * oldj;
            a[UINDEX(lj, jl, k, n, m)] = sint * oldi + cost * oldj;
          }
          a[UINDEX(i, i, k, n, m)] =
              SQUARE(cost) * q + SQUARE(sint) * r - 2 * cost * sint * p;
          a[UINDEX(j, j, k, n, m)] =
              SQUARE(sint) * q + SQUARE(cost) * r + 2 * cost * sint * p;
          a[UINDEX(i, j, k, n, m)] =
              cost * sint * (q - r) + (SQUARE(cost) - SQUARE(sint)) * p;
        }
      }
    }
    fnew = 0.0;
    for (int k = 1; k <= m; k++) {
      for (int i = 1; i <= n; i++) {
        fnew += SQUARE(a[UINDEX(i, i, k, n, m)]);
      }
    }
    if (*verbose == true) {
#ifdef USING_R
      Rprintf("itel %4d fold %15.10f fnew %15.10f\n", *itel, fold, fnew);
#else
      printf("itel %4d fold %15.10f fnew %15.10f\n", *itel, fold, fnew);
#endif
    }
    if (((fnew - fold) < *eps) || (*itel == *itmax))
      break;
    fold = fnew;
    (*itel)++;
  }
  *f = fnew;
  return;
}

void sjacobi(const int *nn, double *a, double *evec, double *fini, double *f,
             int *itel, const int *itmax, const double *eps,
             const bool *verbose, const bool *vectors) {
  int n = *nn;
  double d = 0.0, cost = 0.0, sint = 0.0, u = 0.0, p = 0.0, q = 0.0, r = 0.0,
         piil = 0.0, pijl = 0.0, lbd = 0.0, dd = 0.0, pp = 0.0, dp = 0.0;
  double fold = 0.0, fnew = 0.0, oldi = 0.0, oldj = 0.0;
```

```
*itel = 1;
if (*vectors) {
  for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= n; j++) {
      evec[MINDEX(i, j, n)] = (i == j) ? 1.0 : 0.0;
    }
  }
}
for (int i = 1; i <= n; i++) {
  fold += SQUARE(a[TINDEX(i, i, n)]);
}
*fini = fold;
while (true) {
  for (int j = 1; j <= n - 1; j++) {
    for (int i = j + 1; i <= n; i++) {
      dd = 0.0, pp = 0.0, dp = 0.0;
      p = a[TINDEX(i, j, n)];
      q = a[TINDEX(i, i, n)];
      r = a[TINDEX(j, j, n)];
      d = (q - r) / 2.0;
      dd = SQUARE(d);
      pp = SQUARE(p);
      dp = p * d;
      lbd = ((dd + pp) - sqrt(SQUARE(dd - pp) + 4.0 * SQUARE(dp))) / 2.0;
      u = dp / sqrt(SQUARE(dp) + SQUARE(lbd - pp));
      if ((fabs(dp) < 1e-15) && (pp <= dd)) {
        continue;
      }
      cost = sqrt((1 + u) / 2);
      sint = -sqrt((1 - u) / 2);
      if (*vectors) {
        for (int l = 1; l <= n; l++) {
          piil = evec[MINDEX(l, i, n)];
          pijl = evec[MINDEX(l, j, n)];
          evec[MINDEX(l, i, n)] = cost * piil - sint * pijl;
          evec[MINDEX(l, j, n)] = sint * piil + cost * pijl;
        }
      }
      p = a[TINDEX(i, j, n)];
      q = a[TINDEX(i, i, n)];
      r = a[TINDEX(j, j, n)];
      for (int l = 1; l <= n; l++) {
        if ((l == i) || (l == j))
          continue;
```

```
          int il = IMIN(i, l);
          int li = IMAX(i, l);
          int jl = IMIN(j, l);
          int lj = IMAX(j, l);
          oldi = a[TINDEX(li, il, n)];
          oldj = a[TINDEX(lj, jl, n)];
          a[TINDEX(li, il, n)] = cost * oldi - sint * oldj;
          a[TINDEX(lj, jl, n)] = sint * oldi + cost * oldj;
        }
        a[TINDEX(i, i, n)] =
            SQUARE(cost) * q + SQUARE(sint) * r - 2 * cost * sint * p;
        a[TINDEX(j, j, n)] =
            SQUARE(sint) * q + SQUARE(cost) * r + 2 * cost * sint * p;
        a[TINDEX(i, j, n)] =
            cost * sint * (q - r) + (SQUARE(cost) - SQUARE(sint)) * p;
      }
    }
    fnew = 0.0;
    for (int i = 1; i <= n; i++) {
      fnew += SQUARE(a[TINDEX(i, i, n)]);
    }
    if (*verbose == true) {
#ifdef USING_R
      Rprintf("itel %4d fold %15.10f fnew %15.10f\n", *itel, fold, fnew);
#else
      printf("itel %4d fold %15.10f fnew %15.10f\n", *itel, fold, fnew);
#endif
    }
    if (((fnew - fold) < *eps) || (*itel == *itmax))
      break;
    fold = fnew;
    (*itel)++;
  }
  *f = fnew;
  return;
}

void smpinverse(int *nn, double *a, int *type, int *itmax, double *eps) {
  double fini = 0.0, f = 0.0, s = 0.0, dg = 0.0;
  bool verbose = false, vectors = true;
  int n = *nn, itel = 1;
#ifdef USING_R
  double *evec = (double *)Calloc((size_t)n * n, double);
  double *diag = (double *)Calloc((size_t)n, double);
```

```c
#else
  double *evec = (double *)calloc((size_t)n * n, sizeof(double));
  double *diag = (double *)calloc((size_t)n, sizeof(double));
#endif
  (void)sjacobi(nn, a, evec, &fini, &f, &itel, itmax, eps, &verbose, &vectors);
  for (int i = 1; i <= n; i++){
    dg = a[TINDEX(i, i, n)];
    if (*type == 0) {
      diag[VINDEX(i)] = (fabs(dg) < *eps) ? 0.0 : 1.0 / dg;
    }
    if (*type == 1) {
      diag[VINDEX(i)] = (dg < *eps) ? 0.0 : sqrt (dg);
    }
    if (*type == 2) {
      diag[VINDEX(i)] = (dg < *eps) ? 0.0 : 1.0 / sqrt (dg);
    }
  }
  for (int j = 1; j <= n; j++) {
    for (int i = j; i <= n; i++) {
      s = 0.0;
      for (int k = 1; k <= n; k++) {
        s += evec[MINDEX(i, k, n)] * evec[MINDEX(j, k, n)] * diag[VINDEX(k)];
      }
      a[TINDEX(i, j, n)] = s;
    }
  }
#ifdef USING_R
  (void)Free(evec);
  (void)Free(diag);
#else
  (void)free(evec);
  (void)free(diag);
#endif
  return;
}
```

4 References